KV Cache: How It Works and How to Shrink It

Nov 8, 2026 · @Yazeed Mshayekh

Overview

The KV cache stores the Key and Value vectors of every token already processed, so each new token is computed once instead of re-running the whole sequence. It replaces quadratic recomputation with linear work, but costs GPU memory that grows with context length, number of layers and number of concurrent users.

In production that memory, not the model weights, usually decides how many users one GPU can serve. Three techniques attack it from different sides:

Technique	What it changes	Typical saving
fp8 / int8 KV cache	Bytes per stored value (2 → 1)	2× less cache memory
PagedAttention (vLLM)	How cache memory is allocated	Near-zero waste instead of reserving the maximum context per user
GQA / MQA	How many K/V heads the model has	4–8× (GQA) or up to 32–64× (MQA) less cache

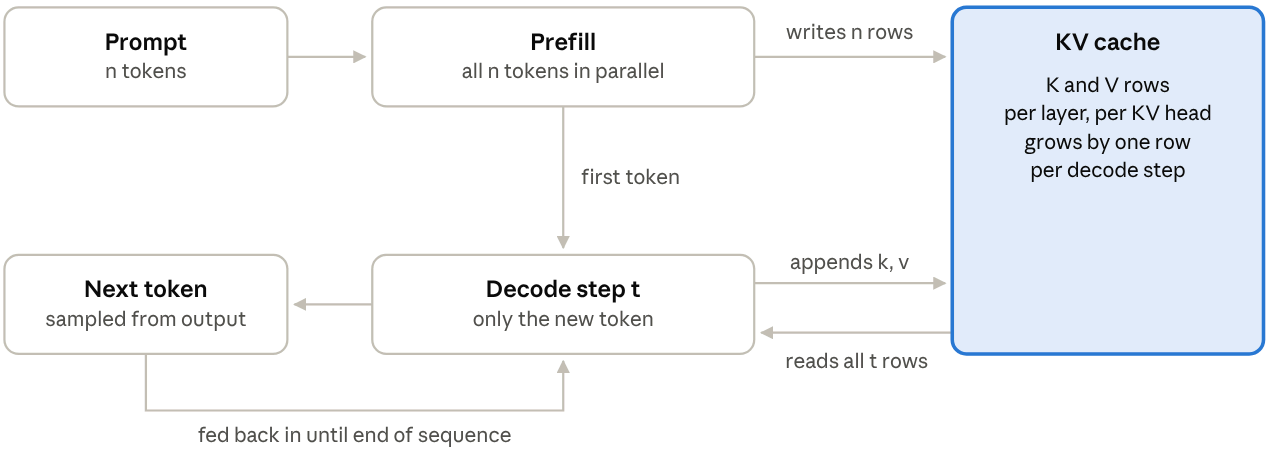
Key terms used throughout:

- **Q, K, V** — the Query, Key and Value vectors each attention head computes for each token.
- **Prefill** — the first pass over the whole prompt, run in parallel.
- **Decode** — generating output one token at a time after prefill.
- **KV head** — one set of K and V projections inside a layer; the cache stores one K and one V vector per token, per KV head, per layer.

How it works, step by step

Generation runs in two phases: one prefill pass fills the cache with the whole prompt, then a decode loop adds one K row and one V row per layer, per KV head, for every new token.

Prefill fills the cache once; each decode step reads it and adds one row



prefill and decode · how the cache is written and read

1. **Prefill (the prompt).** All n prompt tokens go through the model in one parallel pass.
   - In every layer, Q, K and V are computed for all n tokens with one matrix multiply each.
   - Causal attention runs over the full n × n lower-triangular score matrix.
   - The n K rows and n V rows are written into that layer's cache. The Q rows are discarded.
   - The last position's output gives the first generated token.
2. **Decode step t (each new token).** Only the newest token enters the model.
   - In each layer it computes one q, one k and one v vector.
   - k and v are appended to the layer's cache, which now holds t rows.
   - Attention scores q against all t cached keys, then takes the weighted sum of all t cached values.
   - The result goes through the output projection, the FFN and on to the next layer, which repeats the same pattern with its own cache.
3. **Repeat** until an end-of-sequence token or the token limit. The cache grows by exactly one row per layer per KV head each step.
4. **Free** the cache when the request finishes, so the memory can serve another request.

One decode step for one layer and one head, simplified:

```
# x: hidden state of the new token, shape (d_model,)
q = x @ W_q                       # (d_head,)  used now, then discarded
k = x @ W_k                       # (d_head,)
v = x @ W_v                       # (d_head,)

K_cache = concat(K_cache, k)      # (t, d_head)  grows by one row
V_cache = concat(V_cache, v)      # (t, d_head)

scores  = K_cache @ q / sqrt(d_head)   # (t,)  one row of the score matrix
weights = softmax(scores)
out     = weights @ V_cache            # (d_head,)
```

In Hugging Face Transformers this is the `past_key_values` object, enabled by default with `use_cache=True`; serving engines such as vLLM and TensorRT-LLM manage it themselves.

Per decode step at position t	Without a KV cache	With a KV cache
Tokens run through every layer	All t tokens (full forward pass)	1 token
K and V projections computed	t per layer	1 per layer
Attention reads	t keys and values	t keys and values (from memory)
Total K/V projections for n tokens	n(n+1)/2 per layer	n per layer

The cache removes the recomputation, not the attention read itself: each step still reads all t cached rows. That is why long-context decoding is limited by memory bandwidth rather than arithmetic.

Why K and V are cached, but never Q

The new token's output needs only its own query, but it needs the key and value of every token so far. Old queries are never read again, while old keys and values are read by every future token.

The attention output for token t in one head is:

```
o_t = \sum_{i \le t} \mathrm{softmax}_i\!\left( \frac{q_t \cdot k_i}
{\sqrt{d}} \right) v_i
```

Only one query appears in it, q_t. Keys and values appear for every position i from 1 to t.

Old queries have no future use

q_5 existed to compute row 5 of the score matrix, which is token 5′s output. That output became token 5′s hidden state, and the next layer turned it into k_5 and v_5, which are already cached. The causal mask means row 5 never needs recomputing when token 6 arrives, so storing q_5 would spend memory on data nobody reads.

Old keys and values provably never change

This is what makes the cache exact rather than an approximation. It follows by induction over the layers:

1. **Layer 1.** Token i′s input is its embedding plus its position, which depends only on token i.
2. **Layer l to l + 1.** If every hidden state h_j for j ⩽ i depends only on tokens 1…i, then token i′s attention output does too, because the causal mask blocks later positions. The output projection, FFN, residuals and LayerNorm all act on one token at a time, so h_i at layer l + 1 also depends only on tokens 1…i.
3. **Conclusion.** k_i = W_K · h_i and v_i = W_V · h_i are fixed the moment token i is processed, in every layer. Appending new tokens cannot change them.

Why this fails for BERT-style models

Bidirectional encoders let every token attend to tokens on both sides. A new token changes the hidden states, and therefore the K and V, of every earlier token, so nothing cached stays valid. The KV cache depends on causal, decoder-only attention.

Why cache K and V rather than hidden states

Caching each layer′s hidden states would still require two matrix multiplies (W_K and W_V) per past token, per layer, per step, which is exactly the repeated work the cache removes. K and V are the smallest thing the next query actually reads, and with GQA they are smaller than the hidden state anyway.

Memory cost: how big the cache gets

Llama-3-8B in bf16 stores 128 KB of K and V per token, so a single 8K-token conversation holds 1 GB of cache and a 128K-token document holds 16 GB, as much as the model's own weights.

```
\text{KV bytes} = 2 \times L \times H_{kv} \times d_{head} \times b \times T \times B
```

Symbol	Meaning	Llama-3-8B
2	One K and one V	2
L	Layers	32
H_kv	KV heads per layer	8
d_head	Dimension per head (hidden size ÷ query heads)	4096 ÷ 32 = 128
b	Bytes per stored value	2 (bf16/fp16), 1 (fp8/int8)
T	Tokens in the sequence (prompt + generated)	varies
B	Sequences in the batch	varies

The Llama-3-8B values come from its config.json: 32 layers, 32 query heads, 8 KV heads, hidden size 4096.

Worked example: Llama-3-8B, bf16

1. **Per token:** 2 × 32 × 8 × 128 × 2 bytes = 131,072 bytes = 128 KB.
2. **One 8K-token chat:** 128 KB × 8,192 = 1 GB.
3. **One 128K-token document:** 128 KB × 131,072 = 16 GB.
4. **32 users at 8K each:** 1 GB × 32 = 32 GB, double the 16 GB of weights.

Per-token cost across models

Model	Layers × KV heads × head dim	Cache per token (16-bit)	8K context	128K context
Llama-2-7B (full multi-head)	32 × 32 × 128	512 KB	4 GB	64 GB (hypothetical; model supports 4K)
Llama-3-8B (GQA)	32 × 8 × 128	128 KB	1 GB	16 GB
Llama-3-70B (GQA)	80 × 8 × 128	320 KB	2.5 GB	40 GB

Sizes use binary units (1 KB = 1,024 bytes, 1 GB = $1{,}024^3$ bytes). The original Llama 3 supports 8K tokens; Llama 3.1 8B and 70B keep the same layer and head shapes and extend context to 128K, so the 128K figures apply to them.

What it means for a real deployment

Serving Llama-3-8B in bf16 on one 80 GB GPU (A100 or H100) leaves roughly 60 GB for the cache after about 16 GB of weights and a few GB of activations and runtime overhead. 60 GB ÷ 128 KB ≈ 490,000 tokens of cache shared by all active requests.

Context per user	Concurrent users (bf16 cache)	With fp8 cache
4K tokens	~120	~240
8K tokens	~60	~120
32K tokens	~15	~30
128K tokens	~3	~7

These are upper bounds that assume no wasted memory; without PagedAttention a large share of the cache is lost to fragmentation (next sections). The overhead figure is an estimate and varies by engine and batch size.

Optimization 1: fp8 or int8 KV cache

Storing each K and V value in 1 byte instead of 2 halves the cache, so the same GPU holds twice the tokens: about 120 instead of 60 concurrent 8K-token users for Llama-3-8B on an 80 GB GPU. The model weights are untouched; only what is written into the cache changes.

How it works

Values are quantized when they are written to the cache and dequantized when attention reads them:

```
x_q = \mathrm{clamp}\!\left(\mathrm{round}\!\left(\frac{x}{s}\right), -
q_{max}, q_{max}\right), \qquad \hat{x} = x_q \cdot s, \qquad s = \frac{\max
|x|}{q_{max}}
```

- **x** is a K or V value in bf16, **x_q** the stored 8-bit value, **s** the scale, **x̂** the value attention actually uses.
- **q_max** is the largest representable value: 448 for FP8 E4M3, 127 for INT8.
- The scale is the main design choice. It can be **static**, calibrated offline on sample prompts and fixed, or **dynamic**, computed from the data as it arrives.
- Scales can be shared at different **granularities**: one per tensor (per layer, for all of K or all of V), one per KV head, or one per channel or per token. Finer scales track outliers better but store more metadata.

Formats compared

Format	Bits	Layout	Max value	Notes
bf16 / fp16	16	—	very large	Default; the baseline for every number in this doc
FP8 E4M3	8	4 exponent, 3 mantissa bits	±448	More precision, less range; the usual choice for KV caches
FP8 E5M2	8	5 exponent, 2 mantissa bits	±57,344	More range, less precision; no scale tuning needed for outliers

Format	Bits	Layout	Max value	Notes
INT8	8	256 evenly spaced levels	±127 × scale	Precision depends entirely on good scales; runs on any GPU
2–4 bit (research, e.g. KIVI)	2–4	Grouped integer	±(2^(bits−1)) × scale	4–8× smaller; needs careful per-channel and per-token grouping

Accuracy

With calibrated scales, 8-bit KV caches usually change output quality very little, but the loss depends on the model and task, so measure it on your own evaluation set before rolling out.

- **Keys are harder than values.** Keys contain a few outlier channels with much larger magnitude. The KIVI paper found keys quantize best per channel and values per token, and with that split reached 2-bit caches with near-unchanged quality on Llama, Falcon and Mistral.
- **Uncalibrated FP8 is a risk.** In vLLM, `kv_cache_dtype="fp8"` without calibration sets every scale to 1.0; dataset calibration through llm-compressor is the recommended path for best quality (vLLM docs).
- **Some layers are more sensitive.** vLLM notes that sliding-window attention layers are more sensitive to KV quantization and lets you keep them in full precision with `--kv-cache-dtype-skip-layers`.

Speed

Decoding is limited by how fast the GPU can read the cache, and an 8-bit cache is half the bytes to read each step. On top of the capacity gain, attention kernels therefore often run faster for long contexts. With vLLM's Flash Attention 3 backend, attention itself runs in the FP8 domain, with queries quantized too.

How to turn it on

vLLM, server:

```
vllm serve meta-llama/Llama-3.1-8B-Instruct --kv-cache-dtype fp8
```

vLLM, Python:

```
from vllm import LLM
llm = LLM(model="meta-llama/Llama-3.1-8B-Instruct", kv_cache_dtype="fp8")
```

vLLM accepts `fp8`, `fp8_e4m3` (CUDA 11.8+ and ROCm) and `fp8_e5m2` (CUDA 11.8+).

TensorRT-LLM (docs), FP8 KV cache supported from Ampere through Blackwell:

```
from tensorrt_llm import LLM
from tensorrt_llm.llmapi import KvCacheConfig
llm = LLM(model="/path/to/model", kv_cache_config=KvCacheConfig(dtype="fp8"))
```

When to use it

- [ ] You are memory-bound: requests queue because the cache is full, not because compute is busy.
- [ ] You serve long contexts, where the cache dwarfs the weights.
- [ ] You have an evaluation set to confirm quality before and after.
- [ ] You calibrate scales rather than relying on the default of 1.0.

Optimization 2: PagedAttention (vLLM)

PagedAttention stores each request's cache in small fixed-size blocks that are allocated only as the conversation grows, the way an operating system pages virtual memory. Waste drops to at most one partly filled block per request, where earlier servers used only 20.4–38.2% of their cache memory for real token data, and vLLM reported 2–4× higher throughput than FasterTransformer and Orca at the same latency (Kwon et al., SOSP 2023).

The problem: reserving the maximum up front

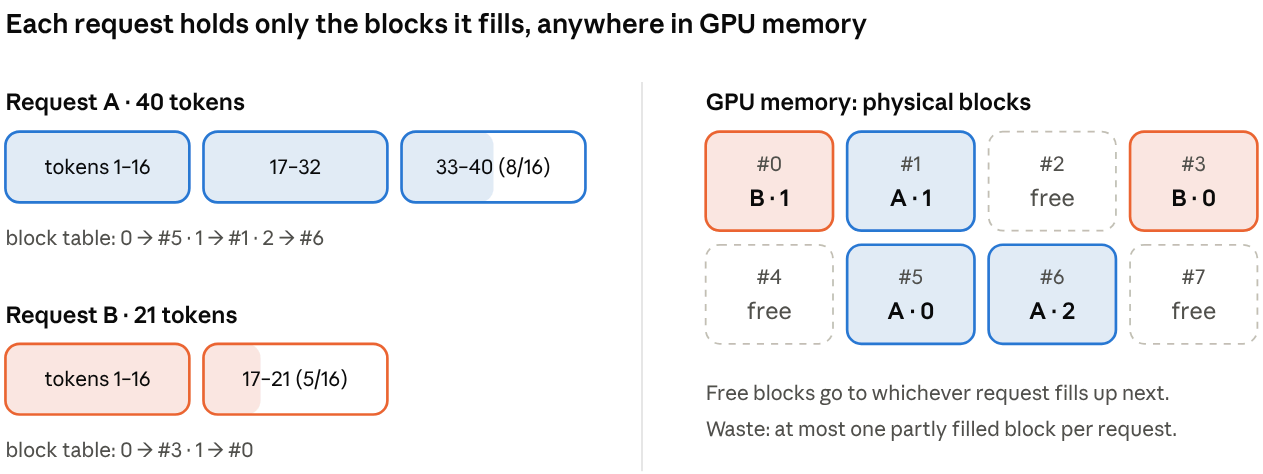
Earlier serving systems gave each request one contiguous buffer sized for the longest sequence it might reach, because they could not know in advance how long the answer would be. That wastes memory in three ways:

Waste	What happens	Why it hurts
Reserved	Slots kept for tokens not generated yet	Memory idle now that another request could use
Internal fragmentation	Buffer sized for the maximum, request ends early	The unused tail is never used at all
External fragmentation	Different-sized buffers leave gaps between them	Free memory exists but no gap is big enough

For Llama-3-8B with an 8K limit, each reservation is 1 GB. If a typical conversation reaches 1,500 tokens, it fills 188 MB of that and wastes 81%. The 60 GB cache budget then serves 60 users instead of about 320.

The idea: pages for the KV cache

1. **Physical blocks.** GPU cache memory is split into equal blocks that each hold the K and V vectors of B consecutive tokens. vLLM's default is B = 16.
2. **Logical blocks.** Each request sees its own cache as a contiguous list of logical blocks: tokens 1–16, 17–32, and so on.
3. **Block table.** A per-request table maps each logical block to a physical block anywhere in memory and records how many of its slots are filled.
4. **Allocate on demand.** A new physical block is taken from the free pool only when the current one is full, so a request never holds more than one partly empty block: at most 15 unused token slots.
5. **Paged attention kernel.** The attention kernel follows the block table and reads keys and values block by block, so the blocks do not need to sit next to each other.
6. **Free immediately.** When a request ends, its blocks go straight back to the pool for the next request.



block tables · 2 requests, 8 physical blocks of 16 tokens

Request A's three logical blocks sit in physical blocks 5, 1 and 6; its last block is half full, which is the only waste it carries.

Sharing blocks with copy-on-write

Because blocks are indirect, several sequences can point at the same physical block.

- **Parallel sampling** (several answers to one prompt) shares the prompt's blocks instead of copying them.
- **Beam search** shares the common history of the beams.
- Each shared block has a **reference count**. When a sequence must write into a shared block, vLLM copies that block first and decrements the original's count, so only the diverging part is ever duplicated.
- On OPT-13B the paper measured 6.1–9.8% memory saved for parallel sampling and 37.6–55.2% for beam search on the Alpaca dataset, and more on ShareGPT.
- The same mechanism lets vLLM reuse the blocks of an identical prompt prefix, such as a shared system prompt, across different requests (prefix caching).

When the cache is full: preemption

If new tokens need blocks and none are free, vLLM pauses some requests and frees their blocks in one of two ways:

- **Swap** the blocks to CPU memory and copy them back later.
- **Recompute** the cache later by treating the already generated tokens as a new prompt.

The paper found the two perform comparably for block sizes of 16 to 64.

Using it

vLLM uses PagedAttention by default. The settings that decide how much cache you get:

Flag	What it controls
`--gpu-memory-utilization`	Fraction of GPU memory vLLM may use for weights, activations and cache (0.9 by default)
`--max-model-len`	Longest sequence allowed; caps the worst case per request
`--max-num-seqs`	Most requests decoded together
`--block-size`	Tokens per block (16 by default)

At startup vLLM logs how many cache blocks it allocated and the maximum concurrency that implies, which is the quickest way to see the effect of fp8 and context-length settings. TensorRT-LLM and Hugging Face TGI use paged KV caches too.

Optimization 3: GQA and MQA

Grouped-query attention (GQA) lets several query heads share one key/value head, so the cache shrinks by the ratio of query heads to KV heads: 4× for Llama-3-8B (32 query heads, 8 KV heads) and 8× for Llama-3-70B (64 and 8). Multi-query attention (MQA) shares a single K/V head across all query heads, cutting the cache 32–64× with more risk to quality.

Unlike the first two optimizations, this one is part of the model's architecture. It is chosen when the model is trained and cannot be switched on at serving time.

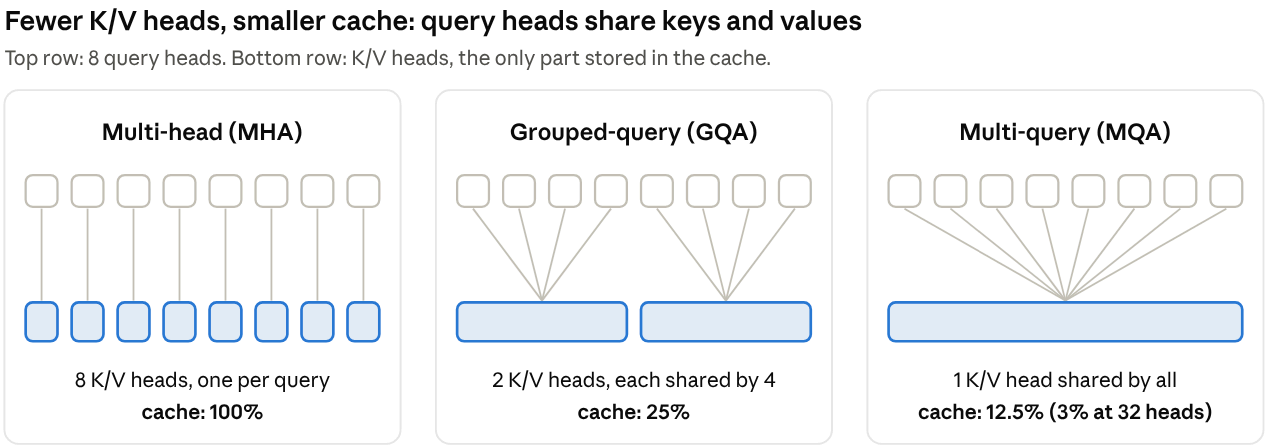
The three designs

All rows assume a model shaped like Llama-3-8B: 32 layers, 32 query heads, head dimension 128, bf16 cache.

Design	KV heads	Query heads per KV head	Cache per token	8K-token chat	Examples
Multi-head (MHA)	32	1	512 KB	4 GB	GPT-2, Llama-2-7B and 13B
Grouped-query (GQA)	8	4	128 KB	1 GB	Llama-2-70B, all Llama 3 sizes, Mistral 7B
Multi-query (MQA)	1	32	16 KB	128 MB	PaLM, Falcon-7B

How it works

1. The H_q query heads are split into G groups of equal size.
2. Each group has one K head and one V head, so the layer computes and caches only G K vectors and G V vectors per token.
3. Every query head still computes its own scores against its group's shared keys, so each head keeps its own attention pattern.
4. MHA is the special case G = H_q; MQA is the special case G = 1.



MHA vs GQA vs MQA · 8 query heads shown

Query head h uses the K and V of group g(h):

$$g(h) = \left\lfloor \frac{h}{H_q / H_{kv}} \right\rfloor, \qquad o_h = \mathrm{softmax}\!\left(\frac{q_h K_{g(h)}^{\top}}{\sqrt{d}}\right) V_{g(h)}$$

Most implementations store only the H_kv heads in the cache and expand them on the fly when attention runs:

```
# k, v from the cache: (batch, n_kv_heads, seq_len, head_dim)
# q for this step:     (batch, n_q_heads, 1, head_dim)
n_rep = n_q_heads // n_kv_heads            # 4 for Llama-3-8B
k = k.repeat_interleave(n_rep, dim=1)      # view as (batch, n_q_heads,
seq_len, head_dim)
v = v.repeat_interleave(n_rep, dim=1)
out = softmax(q @ k.transpose(-2, -1) / head_dim**0.5) @ v
```

Optimized kernels such as FlashAttention read the shared head directly instead of copying it, so the saving applies to both memory size and memory traffic.

Quality trade-off

- **Why GQA holds up.** Each head keeps its own query, so heads can still attend to different tokens. Only what is looked at (K) and what is retrieved (V) is shared inside a group.
- **Why MQA can hurt.** With one K/V head, every query head must read the same keys and values, which limits how differently heads can behave. The GQA paper notes MQA can degrade quality.
- **The middle ground.** The same paper showed GQA reaches quality close to multi-head attention at speed comparable to MQA.

Converting an existing model

The GQA paper also showed how to convert a trained multi-head checkpoint: mean-pool the K and V projection heads within each group, then continue training ("uptraining") with about 5% of the original pre-training compute. Without that extra training, simply pooling heads damages quality.

Effect on speed

Fewer KV heads means fewer bytes to read per decode step and fewer K/V projections to compute. Because long-context decoding is limited by reading the cache, a 4× smaller cache also makes each step faster, not just cheaper in memory.

Newer designs push the same idea further; for example, DeepSeek-V2's multi-head latent attention caches a compressed low-rank vector instead of full K and V heads.

Combining the three

The three techniques multiply because they act on different things: GQA on how many vectors are stored, fp8 on how many bytes each takes, and PagedAttention on how much memory is reserved but never used. Together they take one 80 GB GPU from about 15 to about 650 concurrent conversations in the example below.

Assumptions: a model shaped like Llama-3-8B (32 layers, 32 query heads, head dimension 128), about 60 GB of GPU memory left for the cache, an 8K-token limit, and a typical conversation of 1,500 tokens.

Step	Cache per token	Memory held per conversation	Conversations that fit
Multi-head attention, bf16, full 8K reserved	512 KB	4 GB	~15
+ GQA (8 KV heads)	128 KB	1 GB	~60
+ fp8 KV cache	64 KB	512 MB	~120
+ PagedAttention (only 1,504 tokens held, 94 blocks of 16)	64 KB	94 MB	~650

In practice compute, `--max-num-seqs` and latency targets stop you well before the last figure, but the cache stops being the limit.

Cheat sheet

	fp8 / int8 KV cache	PagedAttention	GQA / MQA
Acts on	Bytes per stored value	How cache memory is allocated	Number of K/V heads
Saving	2× (8-bit), 4–8× (research 2–4 bit)	Removes reserved and fragmented memory	Query heads ÷ KV heads: 4–8× (GQA), up to 64× (MQA)
Quality risk	Small with calibrated scales; test it	None: exact same math	GQA small; MQA larger

	fp8 / int8 KV cache	PagedAttention	GQA / MQA
Who decides	You, at serving time	The serving engine (on in vLLM by default)	The model's designers, at training time
How to get it	`--kv-cache-dtype fp8` in vLLM, `KvCacheConfig(dtype="fp8")` in TensorRT-LLM	Serve with vLLM, TensorRT-LLM or TGI	Pick a GQA model such as Llama 3 or Mistral, or uptrain an MHA checkpoint

Quick capacity check for your own deployment

1. Read `num_hidden_layers`, `num_key_value_heads`, `hidden_size` and `num_attention_heads` from the model's `config.json`.
2. Per-token cache = 2 × layers × KV heads × (hidden size ÷ attention heads) × bytes per value.
3. Cache budget = GPU memory × utilization − weights − a few GB of overhead.
4. Concurrent users ≈ cache budget ÷ (per-token cache × typical tokens per conversation).
5. Compare with vLLM's startup log line on cache blocks and maximum concurrency.

Sources

- Kwon et al., Efficient Memory Management for Large Language Model Serving with PagedAttention, SOSP 2023 — the vLLM paper: memory waste figures, block tables, copy-on-write, preemption.
- Ainslie et al., GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints, 2023 — GQA, MQA trade-offs and uptraining with about 5% of pre-training compute.
- Liu et al., KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache, 2024 — per-channel keys, per-token values, 2-bit caches.
- vLLM, Quantized KV Cache documentation — `--kv-cache-dtype` options, scales and calibration.
- NVIDIA TensorRT-LLM, Quantization documentation — FP8 KV cache support and configuration.

- Meta Llama 3 8B config.json — layer and head counts used in the memory examples.

Architecture facts for other models (Llama-2, Llama-3-70B, Mistral, PaLM, Falcon) and the serving-capacity estimates are from general knowledge and are approximate; check each model's config before relying on them.